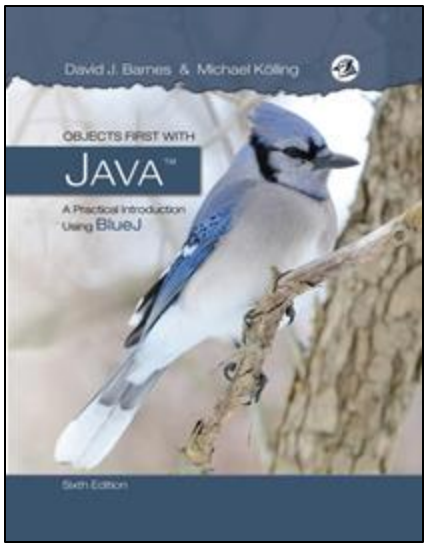
Unidad 01

Objetos & Clases



Esta presentación es una traducción y adaptación de las presentaciones proporcionadas por la editorial Pearson de los capítulos 1 & 2 del libro "Objects First with Java: A Practical Introduction Using BlueJ, 6th Edition" escrito por David J. Barnes y Michael Kolling" © 2017

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Contenidos previos

Las diapositivas siguientes (y los videos enlazados dentro de ellas) **deben** ser visualizados **antes** de la primera sesión presencial de la unidad:
Unidad 01 - Previa 01 - Historia de la POO
Unidad 01 - Previa 02 – Plataforma Java

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Clases y Objetos
- Campos y Estado
- Métodos y Parámetros
- Constructores
- Instancias
- Variables
- Ámbito y vida útil

- Métodos
  - De acceso (get)
  - Mutadores (set)
- Anexos
  - Keywords
  - Tipos de datos primitivos
  - Operadores en Java
  - Sobrecarga de operadores
  - Operadores interesantes
  - Ejercicios

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Características claves de la POO



Clase y objeto

Campo

Método

Tipo de dato

Sentencia de asignación

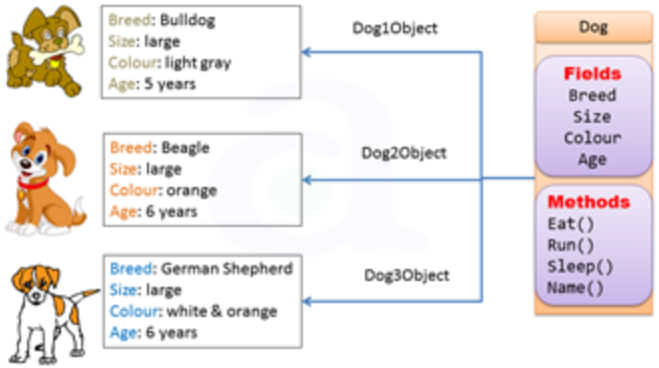
Constructor

Parámetro

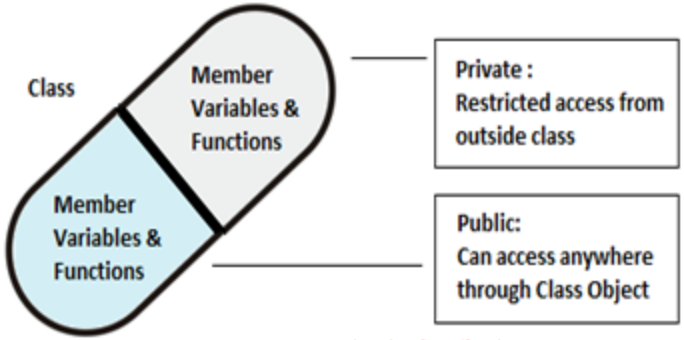
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

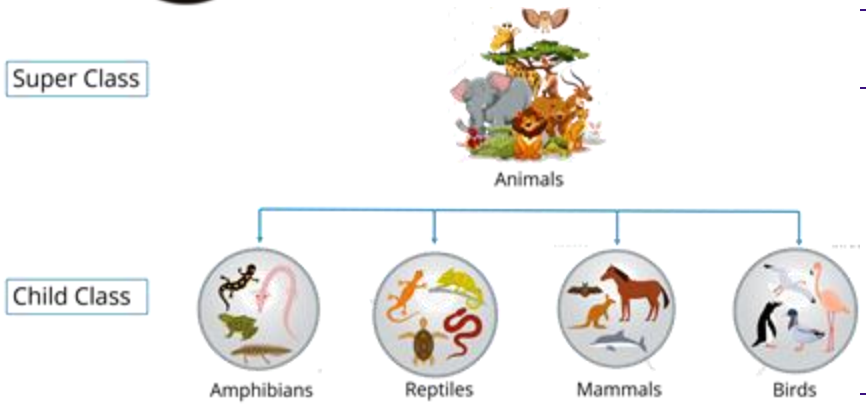
Características claves de la POO



Concepto de clase y objeto: abstracción basada en entidades en lugar de acciones.



Datos encapsulados dentro de las clases: acceso controlado.



Composición, herencia y polimorfismo para facilitar desarrollos.

Conceptos fundamentales

- Clases y Objetos
- Campos y Estado
- Métodos y Parámetros
- Constructores
- Instancias
- Variables
- Ámbito y vida útil

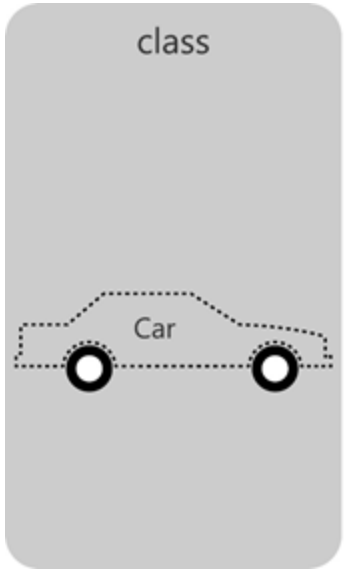
- Métodos
  - De acceso (get)
  - Mutadores (set)
- Anexos
  - Keywords
  - Tipos de datos primitivos
  - Operadores en Java
  - Sobrecarga de operadores
  - Operadores interesantes
  - Ejercicios

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Clases y Objetos

Fundamental para la parte inicial de la asignatura

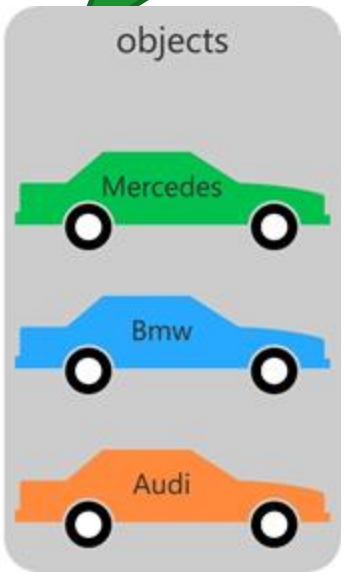


Clase:

- Categoría de tipo o 'cosa'
- Describe el tipo de un objeto
- Como una plantilla o un plano
- Representa a todos los objetos genéricos de un tipo

Objeto:

- Creado a partir de una clase
- Representa **instancias** individuales de una clase
- Pertenece a una clase particular y tiene características individuales.
- Representa una cosa específica' del mundo real o de algún dominio de problema



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Clases y Objetos

Estructura básica de una Clase

```
public class TicketMachine
{
    Contenido interno omitido
}
```

Cabecera de la clase

Envoltorio externo de una clase

```
public class ClassName
{
    Campos
    Constructores
    Métodos
}
```

Contenido interno de una clase

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Clases y Objetos
- Campos y Estado
- Métodos y Parámetros
- Constructores
- Instancias
- Variables
- Ámbito y vida útil

- Métodos
  - De acceso (get)
  - Mutadores (set)
- Anexos
  - Keywords
  - Tipos de datos primitivos
  - Operadores en Java
  - Sobrecarga de operadores
  - Operadores interesantes
  - Ejercicios

Pearson

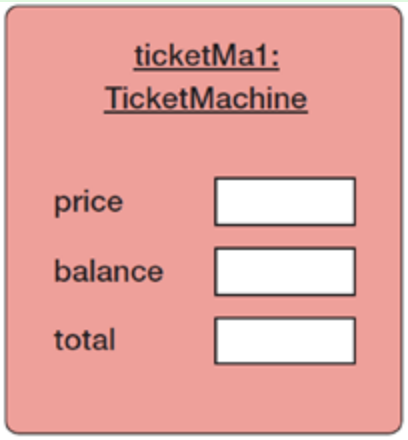
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Campos

Almacenan valores de un objeto

También se llaman variables de instancia

```
12 public class TicketMachine
13 {
14     // The price of a ticket fr
15     private int price;
16     // The amount of money ente
17     private int balance;
18     // The total amount of mone
19     private int total;
20
21     /**
```

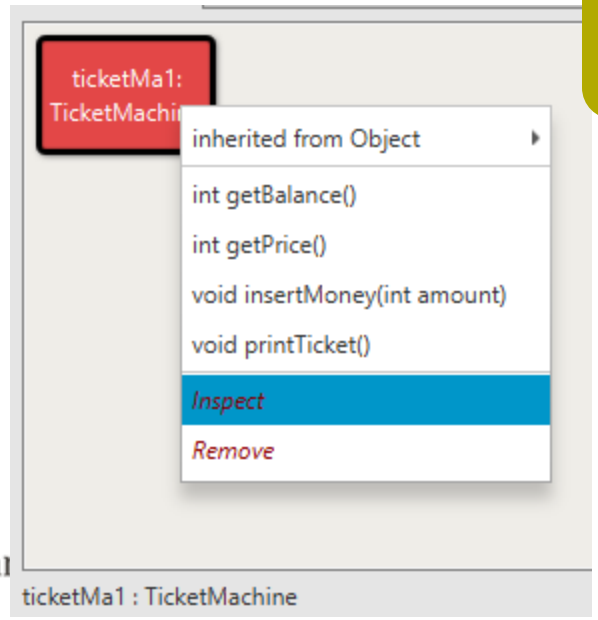


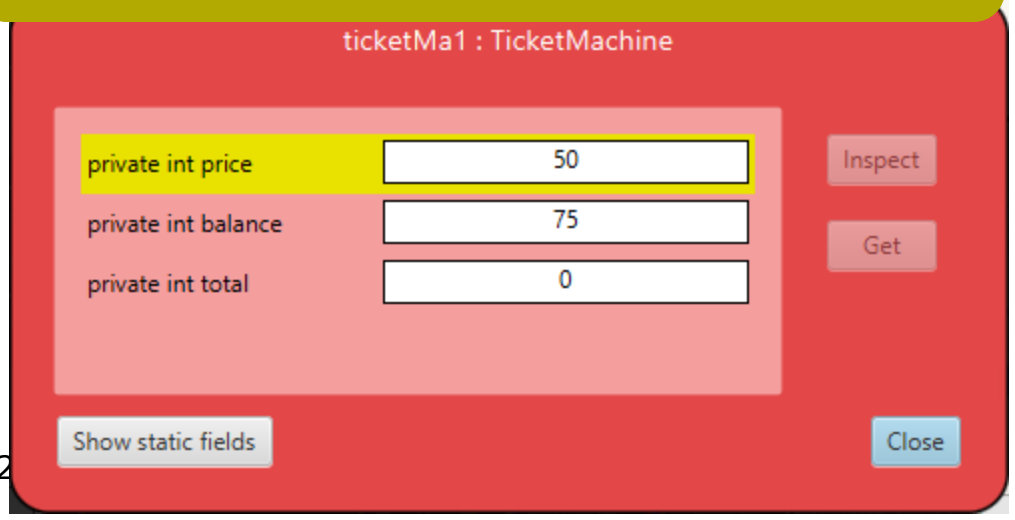
nombre de variable

tipo

modificador de visibilidad

Define el estado de un objeto



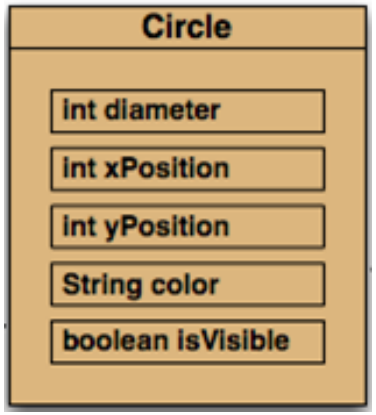


Campos

Atributos

A los valores que almacena un objeto en sus **campos** se los denomina **atributos**

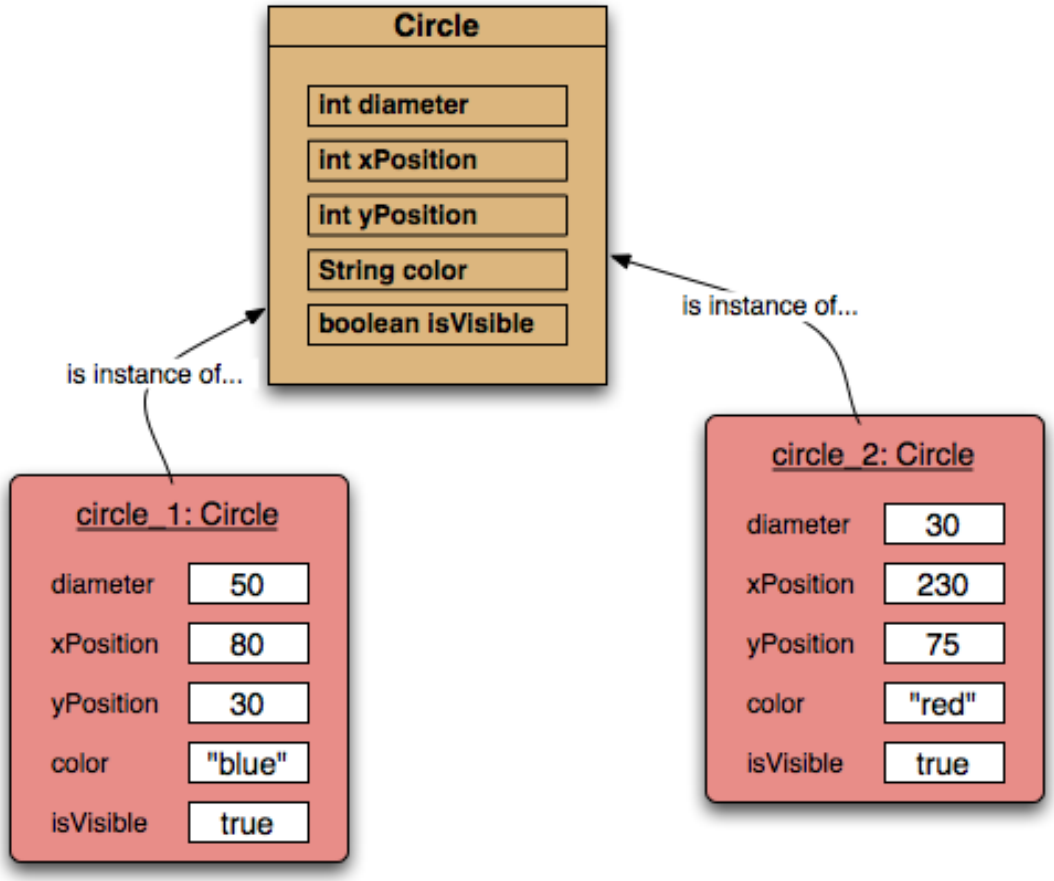
La **clase** define qué campos tiene un objeto pero cada **objeto** almacena su propio conjunto de **valores** (el **estado** del objeto)



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Estado



El **Estado** de un objeto es representado por los valores almacenados en los campos

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Clases y Objetos
- Campos y Estado
- Métodos y Parámetros
- Constructores
- Instancias
- Variables
- Ámbito y vida útil

- Métodos
  - De acceso (get)
  - Mutadores (set)
- Anexos
  - Keywords
  - Tipos de datos primitivos
  - Operadores en Java
  - Sobrecarga de operadores
  - Operadores interesantes
  - Ejercicios

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Métodos y Parámetros

Métodos (verbos)

- Los objetos tienen **operaciones que pueden ser invocadas** sobre un objeto específico.
- Ej: “el coche BMW azul circula”

Parámetros (adverbios)

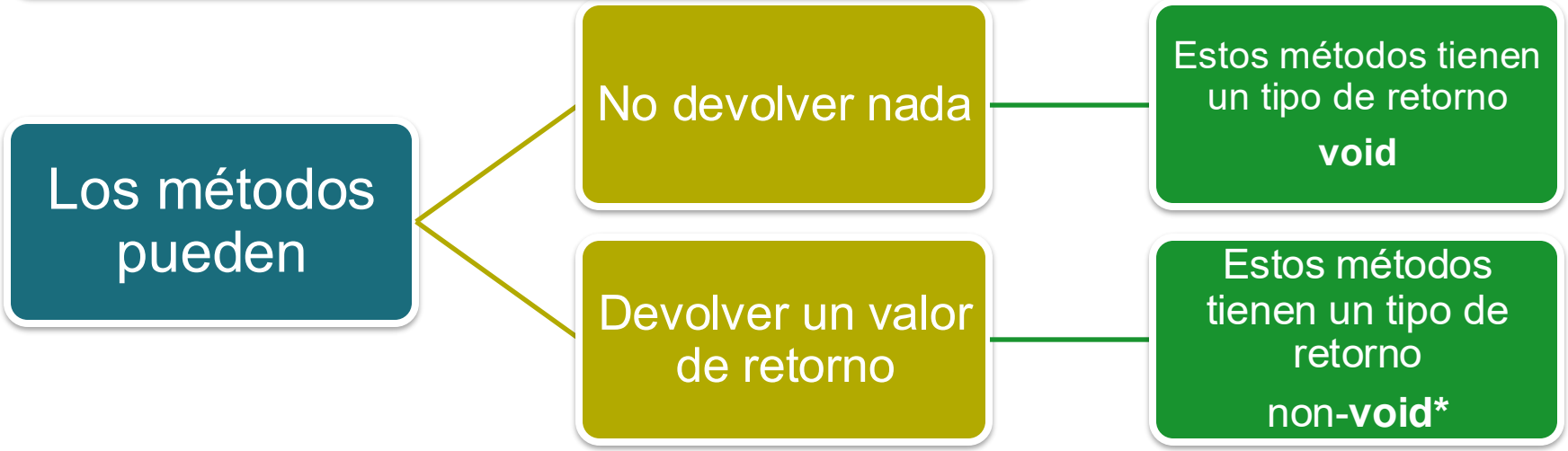
- **Los Métodos** pueden tener **parámetros** para **pasar información adicional** necesaria para ejecutar.
- Introducen variación en el efecto de las llámadas a métodos.
- Los parámetros tienen **tipos** que definen qué tipo de valores puede tomar un parámetro
- Ej: “el coche BMW azul circula durante 10 kilómetros”



Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Métodos y Parámetros

Valor de retorno de un método



*int, String, double, float, un objeto …

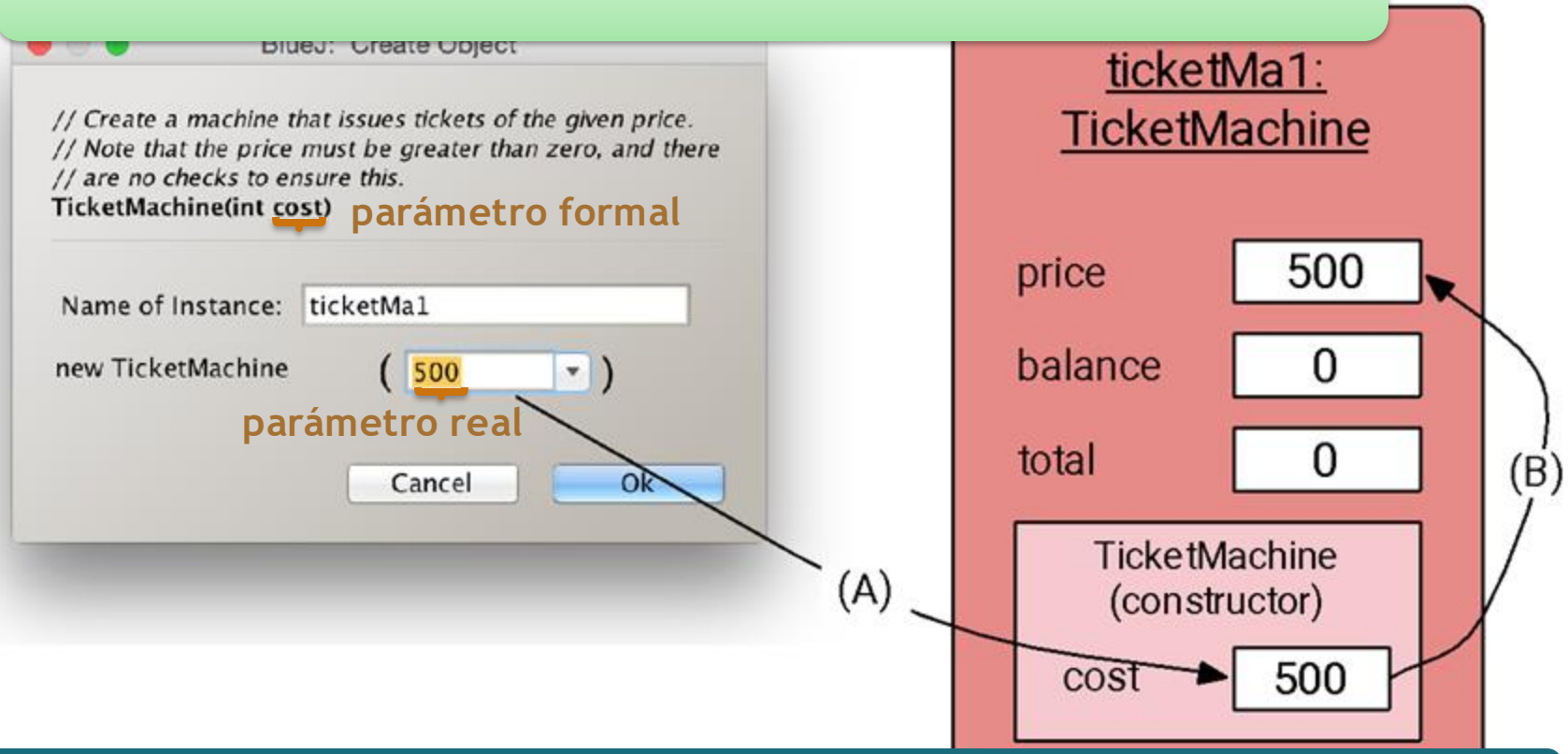
Más sobre esto en la siguiente unidad

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Métodos y Parámetros

Paso de Datos mediante Parámetros



Los Parámetros son otro tipo de variable

(Más sobre variables y parámetros al final de la Unidad)

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Clases y Objetos
- Campos y Estado
- Métodos y Parámetros
- Constructores
- Instancias
- Variables
- Ámbito y vida útil
- Métodos
  - De acceso (get)
  - Mutadores (set)
- Anexos
  - Keywords
  - Tipos de datos primitivos
  - Operadores en Java
  - Sobrecarga de operadores
  - Operadores interesantes
  - Ejercicios

Pearson

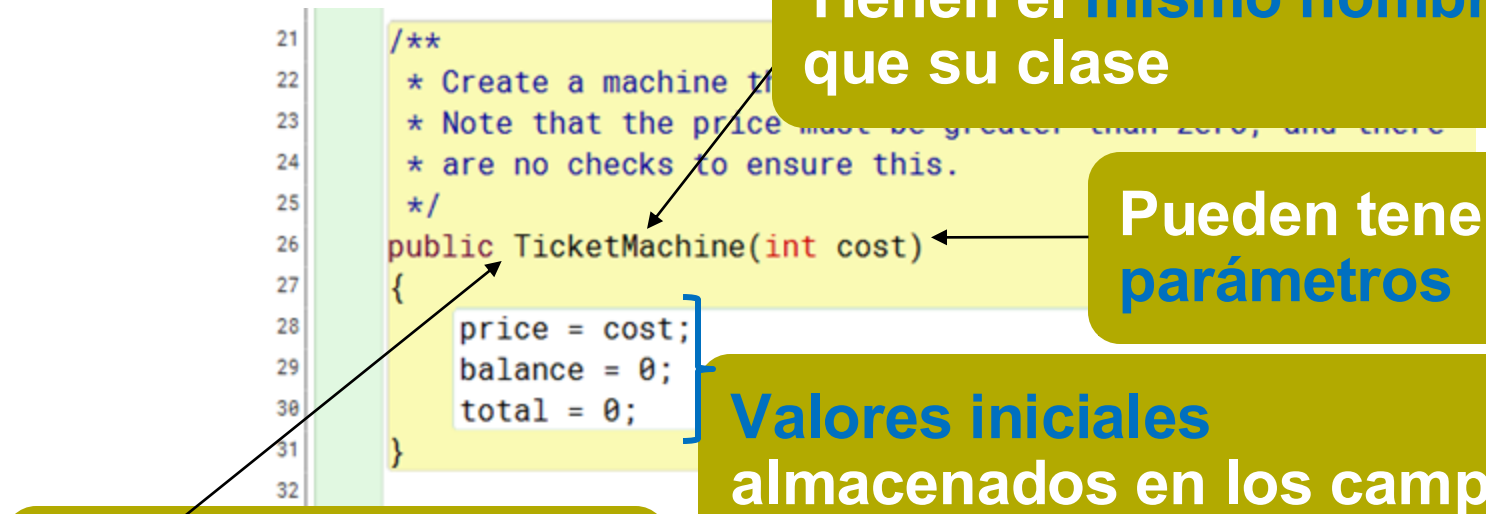
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Constructores

Inicializan un objeto

Puede haber varios

Tienen el mismo nombre que su clase



```
21    /**
22     * Create a machine t
23     * Note that the price
24     * are no checks to ensure this.
25     */
26    public TicketMachine(int cost)
27    {
28        price = cost;
29        balance = 0;
30        total = 0;
31    }
32
```

Pueden tener o no parámetros

Valores iniciales almacenados en los campos

NO tienen un tipo de retorno

Nota: Si no se define un constructor explícito, se utiliza un constructor implícito por defecto sin parámetros que inicializa los campos con valores por defecto.

El constructor por defecto deja de estar disponible tan pronto como se defina al menos un constructor de forma explícita.

Copyri

Conceptos fundamentales

- Clases y Objetos
- Campos y Estado
- Métodos y Parámetros
- Constructores
- Instancias
- Variables
- Ámbito y vida útil

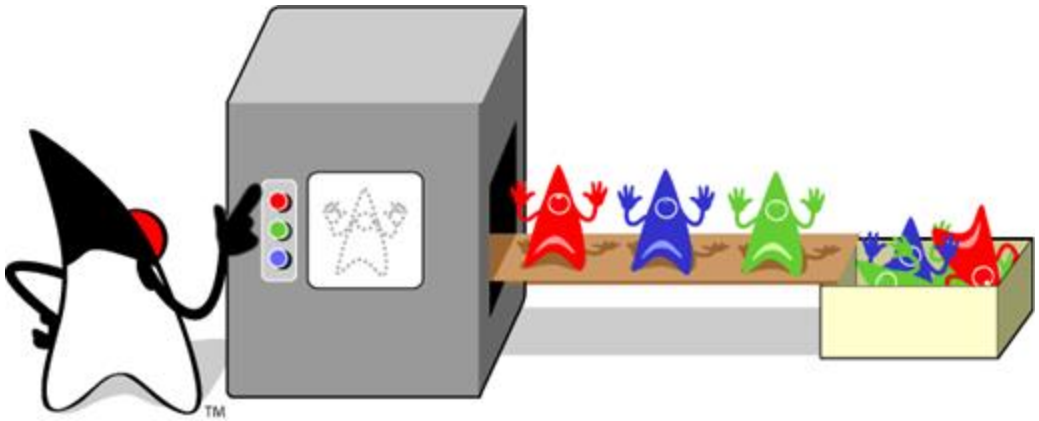
- Métodos
  - De acceso (get)
  - Mutadores (set)
- Anexos
  - Keywords
  - Tipos de datos primitivos
  - Operadores en Java
  - Sobrecarga de operadores
  - Operadores interesantes
  - Ejercicios

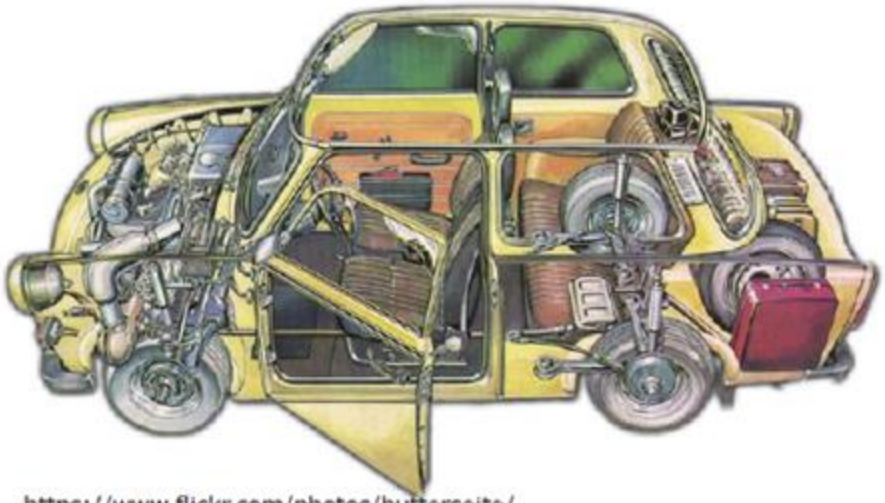
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Se pueden crear muchas **instancias (objetos)** de una sola clase

Instancias





https://www.flickr.com/photos/butterseite/
https://creativecommons.org/licenses/by/2.0/



https://www.flickr.com/photos/pavdw/
https://creativecommons.org/licenses/by/2.0/

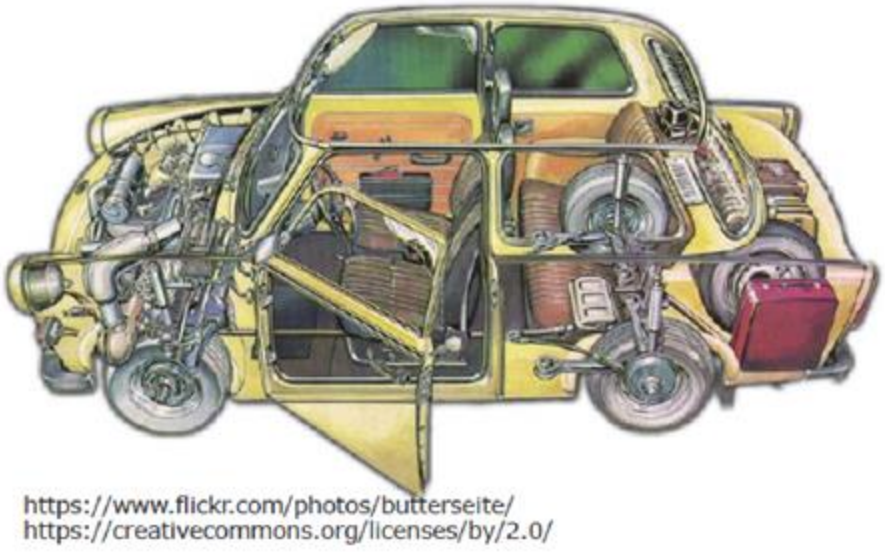
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Instancias

Instanciación mediante constructores

<<nombreClase>> <<identificador>> = new <<constructorClase>>(<<parámetrosInicialización>>);



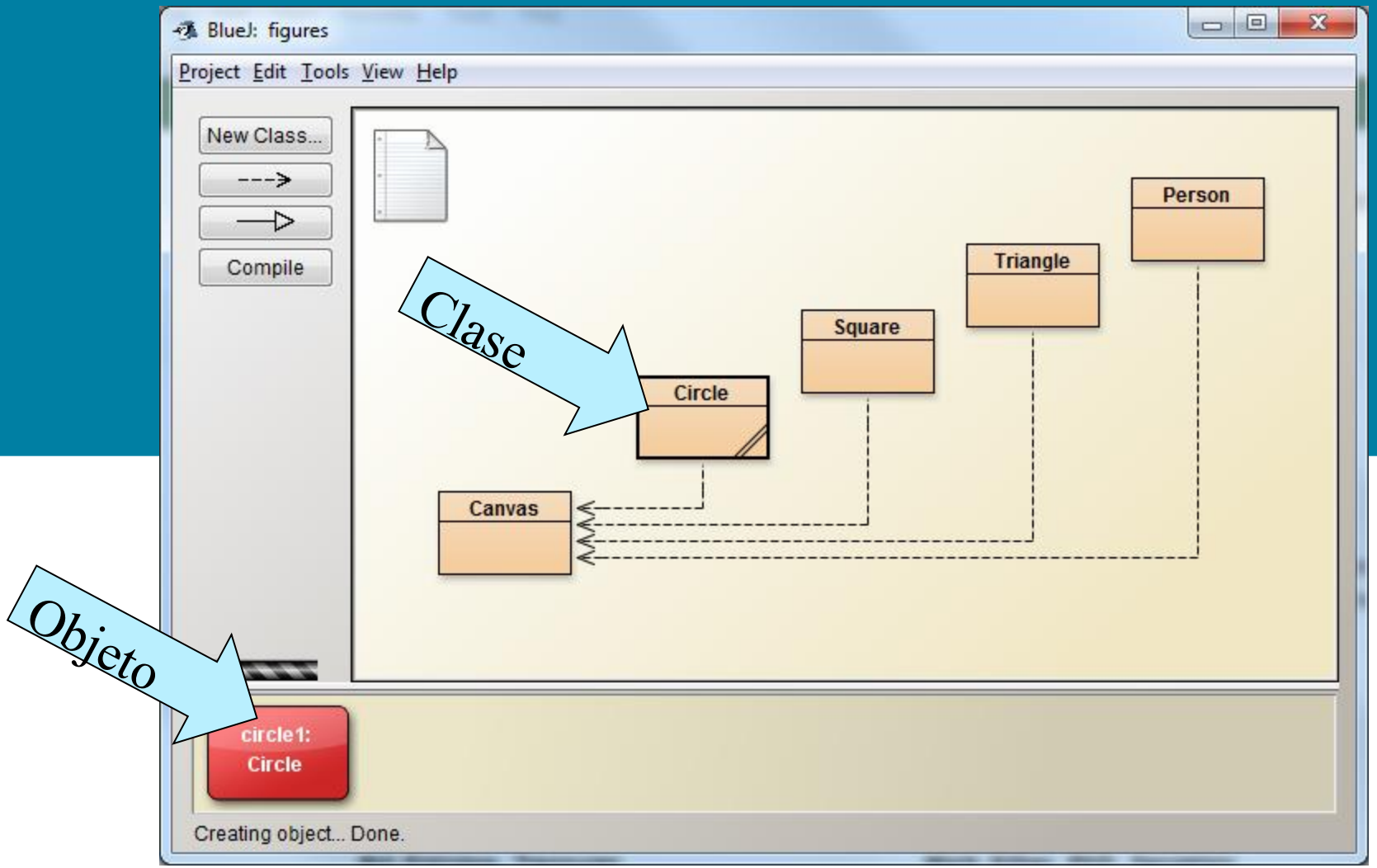



```
CocheAntiguo cocheAntiguoGris = new CocheAntiguo("Gris");
CocheAntiguo cocheAntiguoVerde = new CocheAntiguo("Verde");
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Proyecto "Figures" (Sample Code Unit 1)



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Proyecto "naive-ticket-machine"
(Sample Code Unit 1)

Clase TicketMachine – Una visión del interior

Mirar dentro de una clase nos permite determinar cómo se proporciona o se implementa ese comportamiento

Todas las clases Java tienen una apariencia similar en su interior

Source Code

```
 */
public class TicketMachine
{
    // The price of a ticket from this machine.
    private int price;
    // The amount of money entered by a customer so far.
    private int balance;
    // The total amount of money collected by this machine.
    private int total;

    /**
     * Create a machine that issues tickets of the given price.
     * Note that the price must be greater than zero, and there
     * are no checks to ensure this.
     */
    public TicketMachine(int cost)
    {
        price = cost;
        balance = 0;
        total = 0;
    }

    /**
     * Return the price of a ticket.
     */
    public int getPrice()
    {
        return price;
    }
```

Nota: Los comentarios se insertan en el código fuente de una clase para proporcionar explicaciones a los lectores humanos.

No tienen ningún efecto en la funcionalidad de la clase.

Pearson

Inc. All Rights Reserved

Conceptos fundamentales

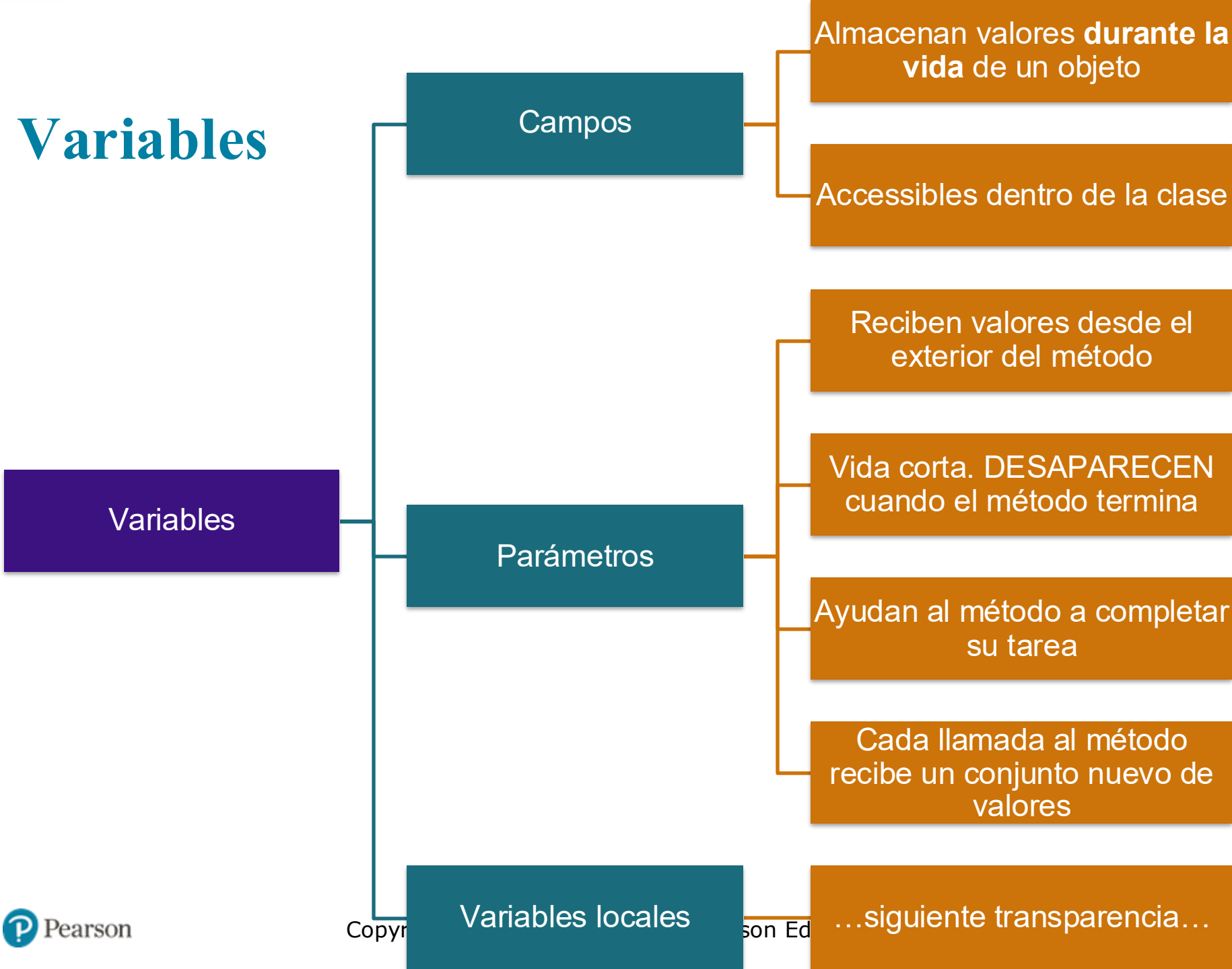
- Clases y Objetos
- Campos y Estado
- Métodos y Parámetros
- Constructores
- Instancias
- Variables
- Ámbito y vida útil

- Métodos
  - De acceso (get)
  - Mutadores (set)
- Anexos
  - Keywords
  - Tipos de datos primitivos
  - Operadores en Java
  - Sobrecarga de operadores
  - Operadores interesantes
  - Ejercicios

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

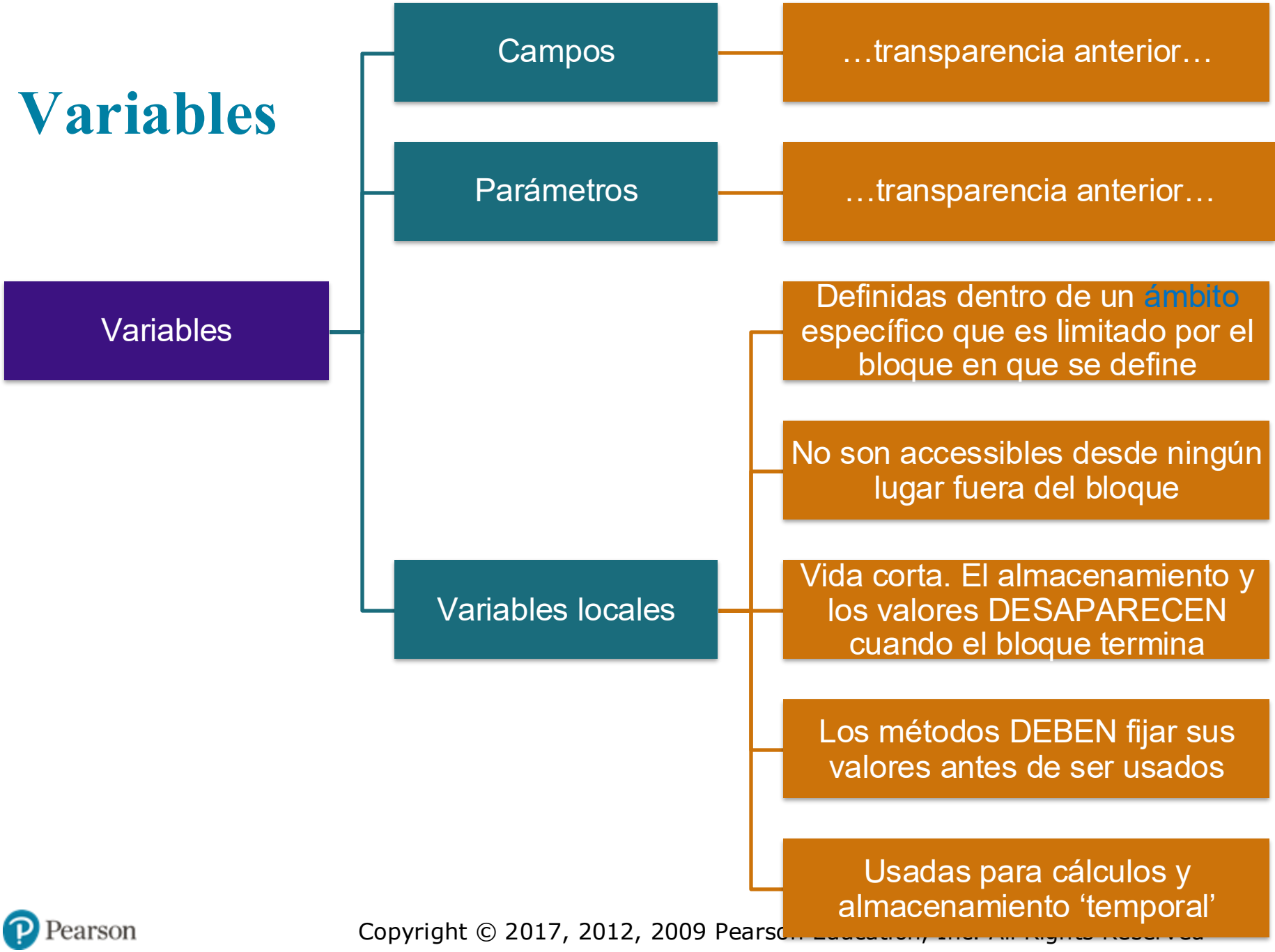
Variables



Pearson

Copyr son Ed

Variables



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Variables

Eligiendo nombres de variables

Elige nombres expresivos

- amount
- price
- age
- name

¡¡Y recuerda!!



Evita una sola letra o nombre crípticos

- w
- t5
- xyz123

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Proyecto "naive-ticket-machine"
(Sample Code Unit 1)

Ejemplo: ¿Cómo escribimos un método que devuelva el dinero sin utilizar?

Intento erróneo

```
public int refundBalance()
{
    // Return the amount left.
    return balance;
    // Clear the balance.
    balance = 0;
}
```

return siempre debe ir en la última sentencia de un método

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Proyecto "naive-ticket-machine"
(Sample Code Unit 1)

Ejemplo: ¿Cómo escribimos un método que devuelva el dinero sin utilizar?

Una variable local

Sin modificador de visibilidad

```
public int refundBalance()
{
    int amountToRefund;
    amountToRefund = balance;
    balance = 0;
    return amountToRefund;
}
```

Podríamos unir declaración & asignación:

```
int amountToRefund = balance;
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Clases y Objetos
- Campos y Estado
- Métodos y Parámetros
- Constructores
- Instancias
- Variables
- Ámbito y vida útil

- Métodos
  - De acceso (get)
  - Mutadores (set)
- Anexos
  - Keywords
  - Tipos de datos primitivos
  - Operadores en Java
  - Sobrecarga de operadores
  - Operadores interesantes
  - Ejercicios

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

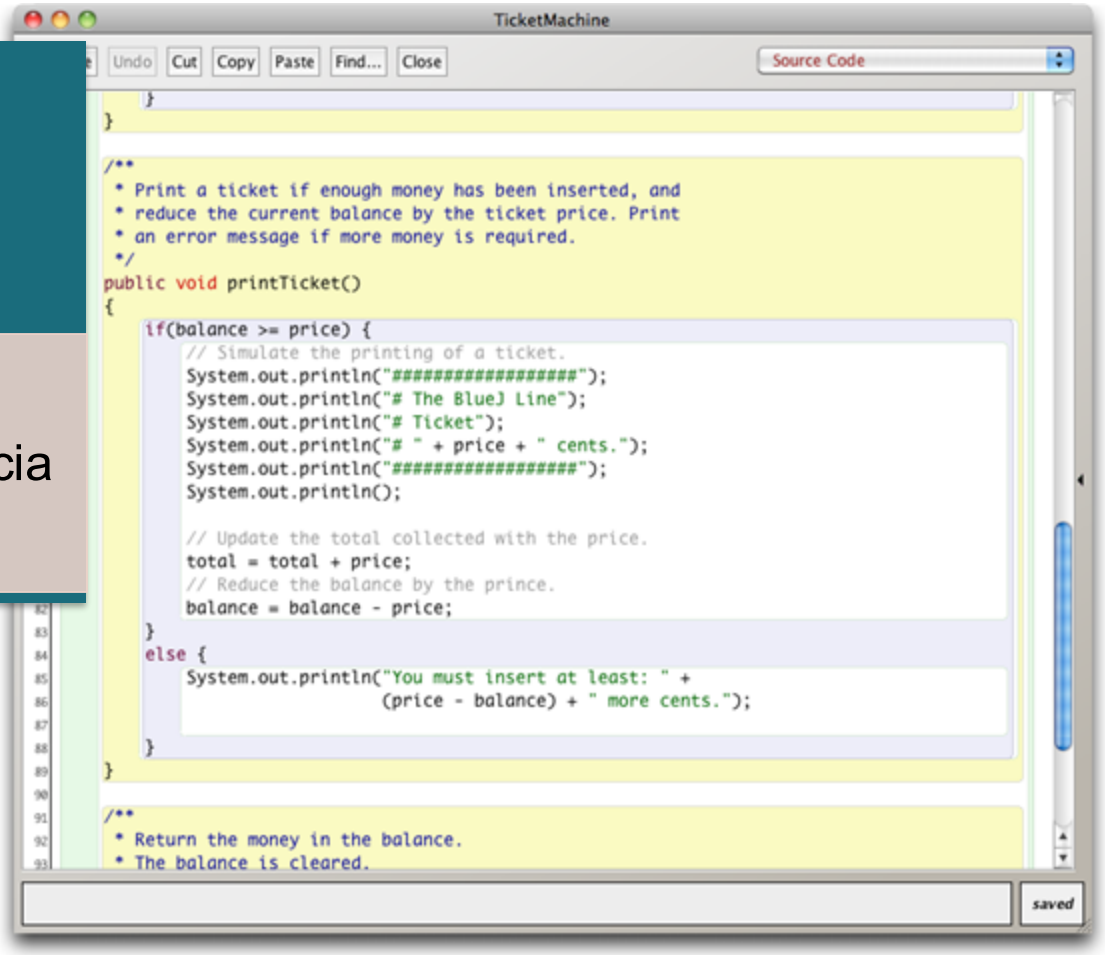
El **ámbito** de una variable define la sección de código fuente desde dónde la variable puede ser accedida

Ámbito

Cada bloque define un nuevo ámbito

Clase	Método	Sentencia

Los ámbitos pueden ser anidados




```
}

/**
 * Print a ticket if enough money has been inserted, and
 * reduce the current balance by the ticket price. Print
 * an error message if more money is required.
 */
public void printTicket()
{
    if(balance >= price) {
        // Simulate the printing of a ticket.
        System.out.println("##################");
        System.out.println("# The BlueJ Line");
        System.out.println("# Ticket");
        System.out.println("# " + price + " cents.");
        System.out.println("##################");
        System.out.println();

        // Update the total collected with the price.
        total = total + price;
        // Reduce the balance by the prince.
        balance = balance - price;
    }
    else {
        System.out.println("You must insert at least: " +
                           (price - balance) + " more cents.");

    }
}

/**
 * Return the money in the balance.
 * The balance is cleared.
```

BlueJ usa distintos colores dentro del código fuente para ayudar a identificar el ámbito

Vida Útil

La **vida útil** de una variable describe cuánto tiempo las variables continúan existiendo antes de ser destruidas

```
public int refundBalance()
{
    int amountToRefund;
    amountToRefund = balance;
    balance = 0;
    return amountToRefund;
}
```

amountToRefund es destruida al salir del método

balance existirá mientras exista el objeto

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ámbito y Vida Útil

Ámbito es estático

Vida útil es dinámica

Campos

- Ámbito: toda la clase donde se declara
- Vida útil: misma de la instancia del objeto

Parámetros

- Ámbito: método/constructor donde se declara
- Vida útil: tiempo de ejecución del método/constructor

Variables locales

- Ámbito: bloque de Código donde se declara
- Vida útil: tiempo de ejecución del bloque de código

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Clases y Objetos
- Campos y Estado
- Métodos y Parámetros
- Constructores
- Instancias
- Variables
- Ámbito y vida útil

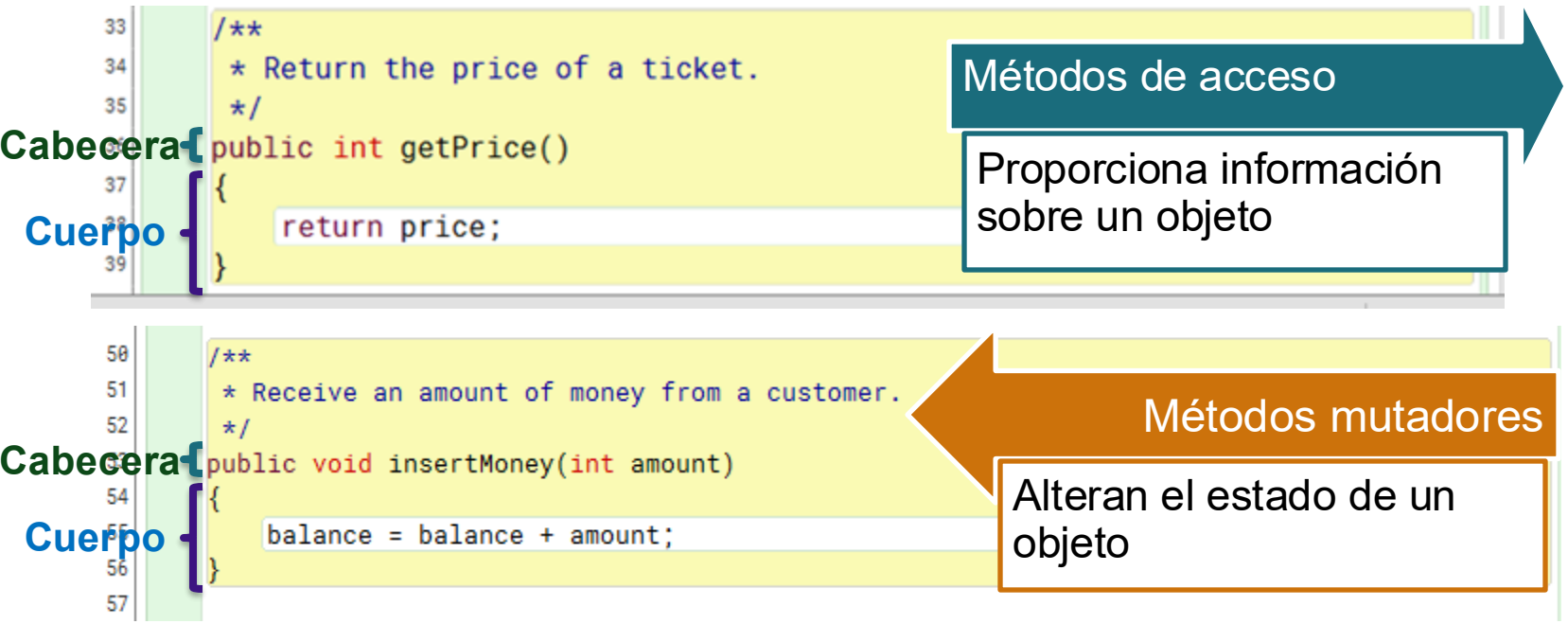
- Métodos
  - De acceso (get)
  - Mutadores (set)
- Anexos
  - Keywords
  - Tipos de datos primitivos
  - Operadores en Java
  - Sobrecarga de operadores
  - Operadores interesantes
  - Ejercicios

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Métodos

Implementan el **comportamiento** de los objetos.



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Clases y Objetos
- Campos y Estado
- Métodos y Parámetros
- Constructores
- Instancias
- Variables
- Ámbito y vida útil

- Métodos
  - De acceso (get)
  - Mutadores (set)
- Anexos
  - Keywords
  - Tipos de datos primitivos
  - Operadores en Java
  - Sobrecarga de operadores
  - Operadores interesantes
  - Ejercicios

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Métodos de acceso (`get`)

Devuelven información acerca del **estado** de un objeto

Su tipo de retorno siempre es distinto de **void**

Devuelve un valor (**resultado**) del tipo especificado en la cabecera

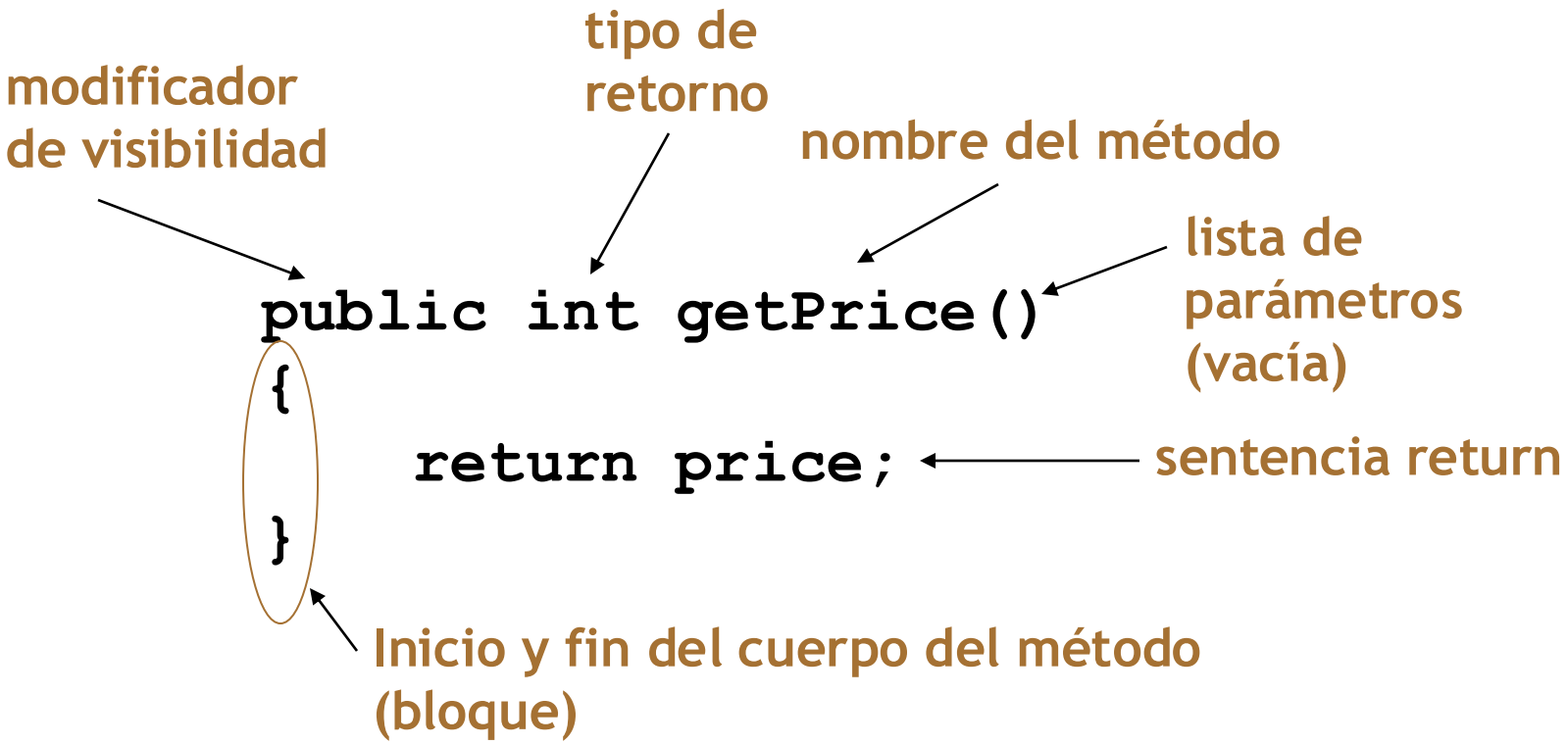
Debe contener una sentencia **return** para devolver el valor

Recuerda: devolver un valor **no** es imprimirlo!

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Métodos de acceso (get)



Cualquier conjunto de declaraciones y sentencias entre un par de llaves se llama *bloque*.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Clases y Objetos
- Campos y Estado
- Métodos y Parámetros
- Constructores
- Instancias
- Variables
- Ámbito y vida útil

- Métodos
  - De acceso (get)
  - Mutadores (set)
- Anexos
  - Keywords
  - Tipos de datos primitivos
  - Operadores en Java
  - Sobrecarga de operadores
  - Operadores interesantes
  - Ejercicios

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Métodos mutadores (`set`)

Cambian (mutan) el **estado** de un objeto

Se consigue cambiando el valor de
uno o más campos

Dan soporte a la **encapsulación** protegiendo los campos

A menudo **reciben parámetros**

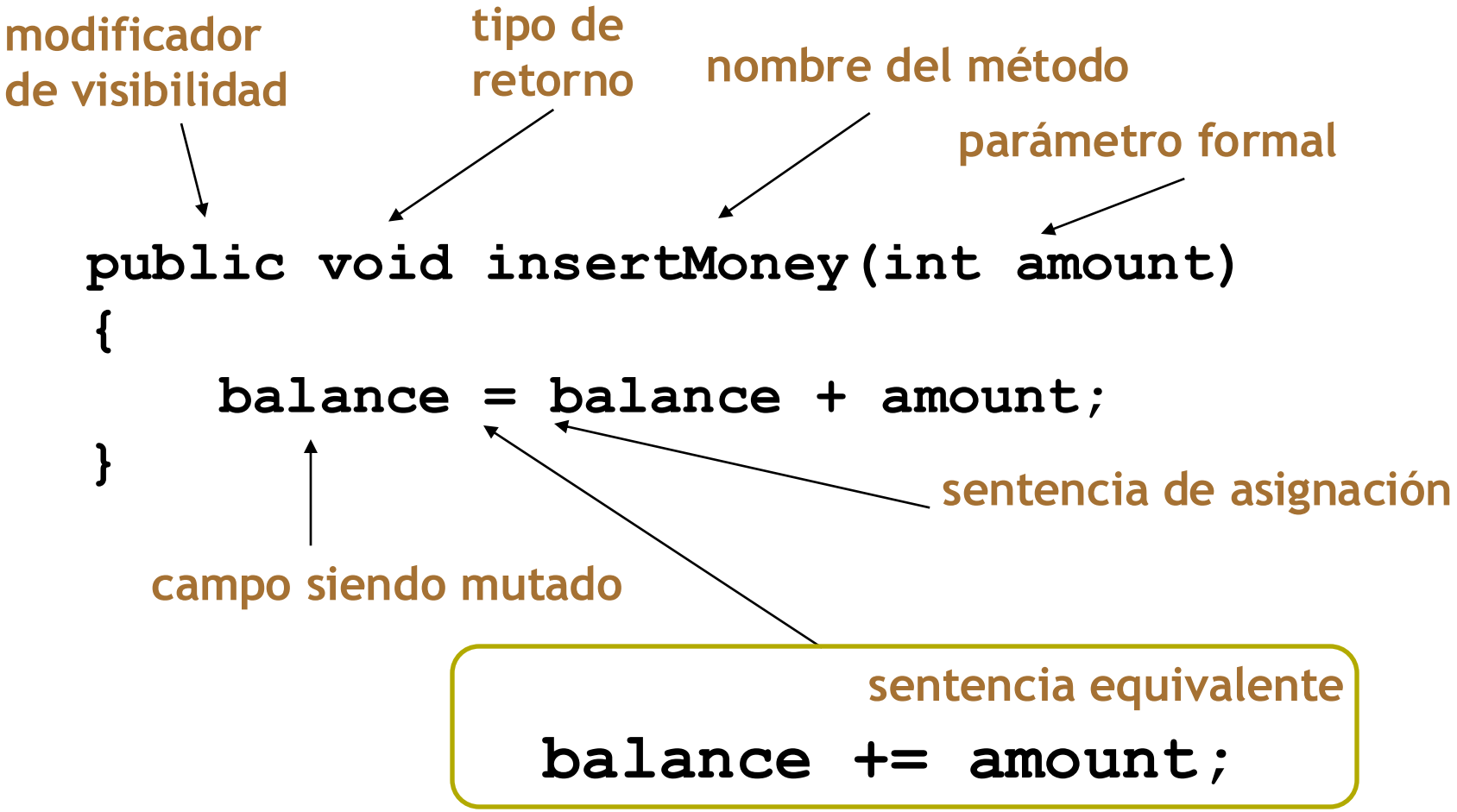
Normalmente contienen
una o más sentencias de asignación

Una asignación podría ser rechazada si al comprobar el parámetro este no es válido

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Métodos mutadores (`set`)



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Métodos set & get de un Campo

```
public class NombreClase
{
 private tipoCampo1 campo1;

 Otros campos
 Constructor/es
 Otros Métodos

 public tipoCampo1 getCampo1(){
     return  campo1;
}

public void setCampo1 (tipoCampo1 nuevoCampo1){
    campo1 = nuevoCampo1;
 }
 }
```

Los campos de una clase a menudo tienen métodos **get & set**

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Clases y Objetos
- Campos y Estado
- Métodos y Parámetros
- Constructores
- Instancias
- Variables
- Ámbito y vida útil
- Métodos
  - De acceso (get)
  - Mutadores (set)
- Anexos
  - Keywords
  - Tipos de datos primitivos
  - Operadores en Java
  - Sobrecarga de operadores
  - Operadores interesantes
  - Ejercicios

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Anexos

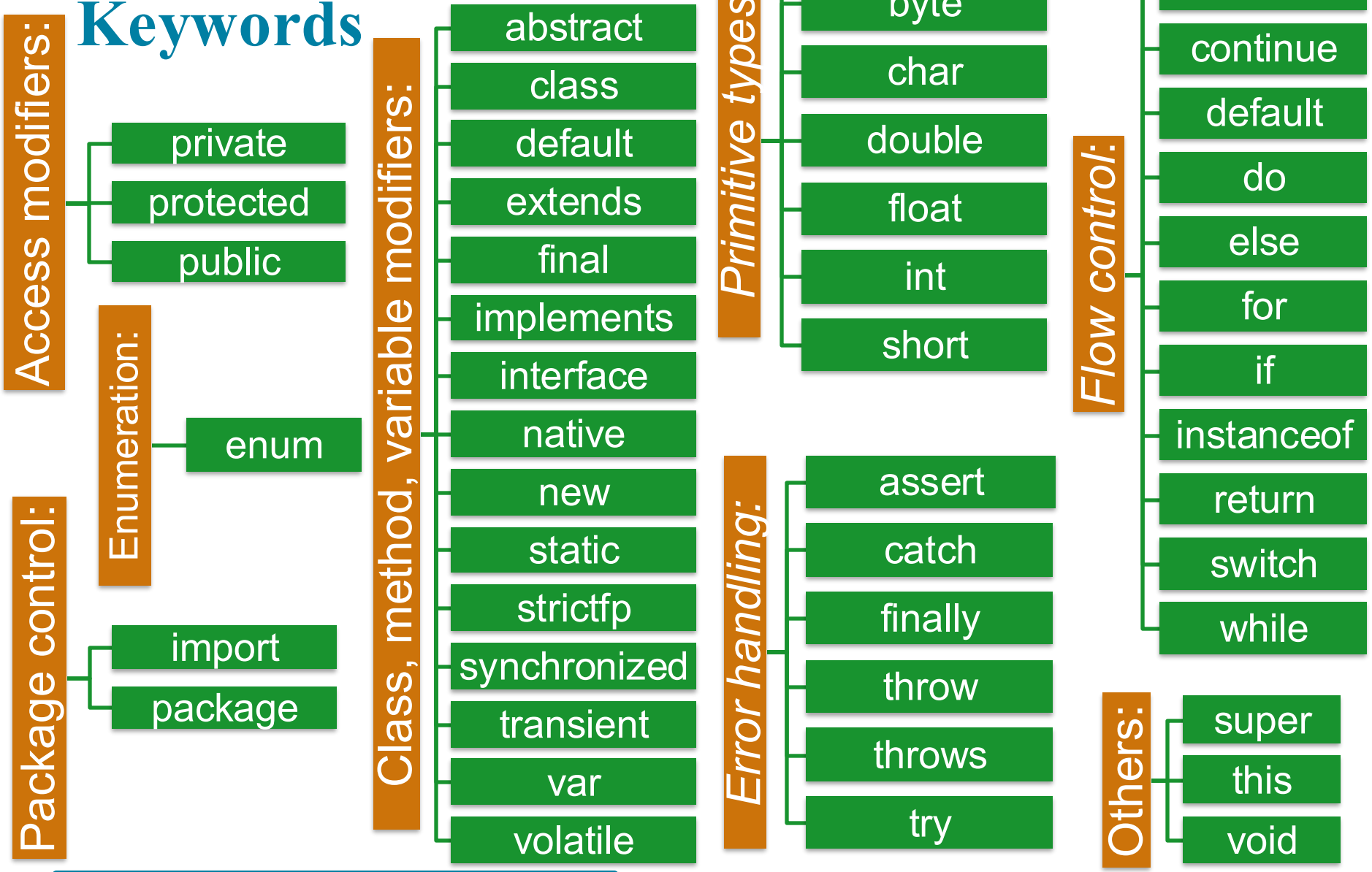
Las diapositivas siguientes **deben** ser leídas **antes** de la siguiente sesión presencial y de laboratorio de la asignatura:

Unidad 01 – Anexo 01 – Keywords
Unidad 01 – Anexo 02 – Tipos de datos primitivos
Unidad 01 – Anexo 03 – Operadores en Java
Unidad 01 – Anexo 04 – Sobrecarga de operadores
Unidad 01 – Anexo 05 – Operadores interesantes
Unidad 01 – Anexo 06 – Ejercicios

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Anexo 1

Keywords



Recuerda: siempre en minúsculas

Anexo 2

Tipos de datos primitivos

Tipo de dato	Descripción:	Tamaño:	Por defecto:
boolean	Pueden ser **true** (correcto) o **false** (incorrecto)	1 bit	false
byte	Enteros entre *-128* y *127*	8 bit	0
char	Carácteres Unicode p.e. *'a'*, *'\u0041'*, *'ü'*, o *'\n'*	16 bit	\u0000
short	Enteros entre *-32768* y *32767*	16 bit	0
int	Enteros entre *-2147483648* y *2147483647*	32 bit	0
long	Enteros entre *-9223372036854775808* y *9223372036854775807*	64 bit	0
float	Números de Punto Flotante entre *-3.4028235E38* y *3.4028235E38*	32 bit	0.0
double	Números de Punto Flotante entre *-1.79769313486231570E308* y *1.79769313486231570E308*	64 bit	0.0

Más sobre tipos de datos primitivos

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Anexo 3

Operadores en Java

Aritméticos

Operador	Descripción:
+	Suma
-	Resta
*	Multiplicación
/	División
%	Módulo, Resto de la división
-	Signo algebraico negativo
- -	Decrementar
++	Incrementar

Formas abreviadas

Operador	Descripción:
+=	Suma el operando de la derecha al operando de la izquierda y asigna el resultado al operando de la izquierda
-=	Resta el operando de la derecha al operando de la izquierda y asigna el resultado al operando de la izquierda
*=	Multiplica el operando de la derecha por el operando de la izquierda y asigna el resultado al operando de la izquierda
/=	Divide el operando de la izquierda entre el operando de la derecha y asigna el resultado al operando de la izquierda
--	Decrementa el operando de la izquierda en 1
++	Incrementa el operando de la izquierda en 1

Pearson

Copyright © 2017,

Anexo 3
Operadores en Java

Comparadores

Operador	Descripción:
==	Igualdad de tipos de dato primitivos (p.ej. int, double, boolean, . . .)
!=	Distinto
>	Mayor que
<	Menor que
>=	Mayor o igual
<=	Menor o igual
equals()	Igualdad de tipos de dato objeto (p.ej. String, . . .)

Lógicos

Operador	Descripción:
&&	AND
||	OR
!	NOT

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Anexo 4

Sobrecarga de operadores

- 4 + 5
  9

El operador (+) representa en este caso la operación suma

- "wind" + "ow"
  "window"
- "Result: " + 6
  "Result: 6"
- "# " + price + " cents"
  "# 500 cents"

El operador (+) representa en este caso concatenación de String.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Anexo 4

Sobrecarga de operadores

- result = op1 + op2;

El operador (=) representa en este caso la asignación

- if (result == 0) …

El operador (==) representa en este caso la comparación de valores

En este caso no es sobrecarga de operadores si no que el operador (=) y el operador (==) tienen usos diferentes

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Anexo 5

Operadores interesantes

Módulo

- El operador de 'división' (/) aplicado a números enteros devuelve el **resultado** de una **división entera**.
- El operador 'modulo' (%) devuelve el **resto** de una **división entera**.
  - P. ej.
    - De forma general:

      17 / 5 Da como resultado 3 y resto 2

En Java:

```
17 / 5 == 3
17 % 5 == 2
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

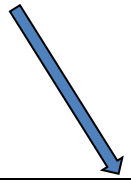
Anexo 5
Operadores interesantes

Condicional

- Elige entre dos valores:

```
condition ? value1 : value2
```

```
for(int i = 1; i < 10; i++) {
    System.out.println(number%2==0?"even":"odd");
}
```



```
if(number%2==0) {
    System.out.println("even");
}
else {
    System.out.println("odd");
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejercicios para pensar

Trata de resolverlos de forma individual antes de pasar a la siguiente diapositiva para ver la solución

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejercicio 1

¿Qué es incorrecto?
(¡hay 5 errores!)

```
public class CokeMachine
{
private price;

public CokeMachine()
{
    price = 300
}

public int getPrice
{
    return Price;
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Solución Ejercicio 1

¿Qué es incorrecto?
(¡hay 5 errores!)

```
public class CokeMachine
{
    int
private price;

public CokeMachine()
{
    price = 300;
}

public int getPrice()
{
    return Price;
}
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejercicio 2

¿Cuál es el resultado?

```
System.out.println(5 + 6 + "hello");

System.out.println("hello" + 5 + 6);
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Solución Ejercicio 2

¿Cuál es el resultado?

```
System.out.println(5 + 6 + "hello");
```

```
11hello
```

```
System.out.println("hello" + 5 + 6);
```

```
hello56
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejercicio 3

¿Cuál es el resultado?

```
8 % 3
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Solución Ejercicio 3

¿Cuál es el resultado?

```
8 % 3
```

2

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejercicio 4

Para un entero `n` >= 0, cuáles son los posibles resultados de:

$n \% 5$

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Solución Ejercicio 4

Para un entero `n` >= 0, cuáles son los posibles resultados de:

$n \,\%\, 5$

`0, 1, 2, 3 y 4`

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Clases y Objetos
- Campos y Estado
- Métodos y Parámetros
- Constructores
- Instancias
- Variables
- Ámbito y vida útil

- Métodos
  - De acceso (get)
  - Mutadores (set)
- Anexos
  - Keywords
  - Tipos de datos primitivos
  - Operadores en Java
  - Sobrecarga de operadores
  - Operadores interesantes
  - Ejercicios

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Otra Bibliografía

- Tony Gauvin - Introduction to Programming. Presentations. University of Maine. Fort Kent.
- Christiane Hagedorn, Ann Katrin Kuessner, Thomas Staubitz, Ralf Teusner. Object-Oriented Programming in Java. OpenSAP Courses

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Otras fuentes de imágenes:

1. https://de.wikipedia.org/wiki/Datei:History_of_object-oriented_programming_languages.svg
2. https://www.atnyla.com/library/java/class-object-and-methods-in-Java/class-and-objects-in-java/class-and-object-in-java-6.PNG
3. https://www.cpp.thiyagaraaj.com/cms/assets/cimages/encapsulation.png
4. https://www.cleanpng.com/png-inheritance-object-oriented-programming-class-java-3757396/preview.html
5. https://docs.oracle.com/javase/tutorial/figures/getStarted/getStarted-jvm.gif
6. https://docs.oracle.com/javase/tutorial/figures/getStarted/helloWorld.gif

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved